

Jane Street “Can you reverse engineer an ASIC?” Puzzle Solution

David Garner, August 2026

This document describes how I solved the Jane Street “Can you reverse engineer an ASIC?” puzzle posted on the Jane Street blog on 5 August 2026 [1].

Toolset Choice

Firstly I had to decide whether to adopt the open-source tools that are maintained and promoted by the open-source silicon chip design community, or the professional (and expensive!) software provided by Cadence Design Systems, which I already had straightforward access to. The Jane Street blog post [1] hinted heavily towards using open-source tools such as *klayout* for viewing layouts and *Surfer* for viewing waveforms, suggesting that this was likely the best approach. However, after some deliberation, I decided to proceed with Cadence tools in the first instance due to easy access and familiarity – whilst keeping open the possibility of switching to the open-source tools later.

Unpacking the Design

Process Design Kit Choice

The blog post did not state what process the GDSII related to other than saying that the design uses an “open-source PDK”. This narrowed down options considerably. I was already aware of the Skywater 130nm PDK (sky130) being open-source due to the interest that eFabless (now ChipFoundry following aquisition) garnered some years back by offering free MPW runs, etc. A quick Google search revealed that there were a couple of others as well but I decided that the Skywater 130nm PDK was the most likely and proceeded on that assumption.

The next problem to address was whether or not the sky130 PDK would work with the Cadence toolset – foundry PDKs are notoriously variable in how compliant they are with various IC design tools and a degree of “hacking” is generally required, with which I am all too familiar with my day job.

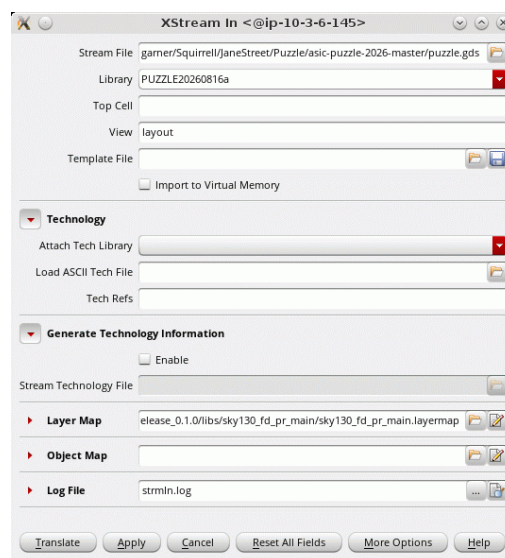
In the end, I discovered that there was a Cadence version of the sky130 PDK available, to people who had a Support Account with Cadence [3]. This made things considerably easier compared to the alternative of trying to get the open-source PDK working with Cadence tools, so I proceeded to download and install the Cadence PDK. Installation was simply a case of untarring a few files and running some scripts.

I created a fresh user directory for myself and had a quick snoop around the PDK using the Cadence “Virtuoso” software (essentially, the cockpit from which various applications are launched including a schematic and layout editor) and could see the *sky130_fd_pr_main* library which contained all the “primitive” analog devices such as transistors, resistors, capacitors etc and also the Cadence libraries of standard cells, *sky130_scl_9T**. I noted that the standard cells – as well as obviously having Verilog models associated with them – contained symbol and schematic views as well, such that they can easily be used by designers with more of an analog-centric background, which was welcomed. I will have more to say about the Cadence standard cell libraries later.

Streaming In the GDSII

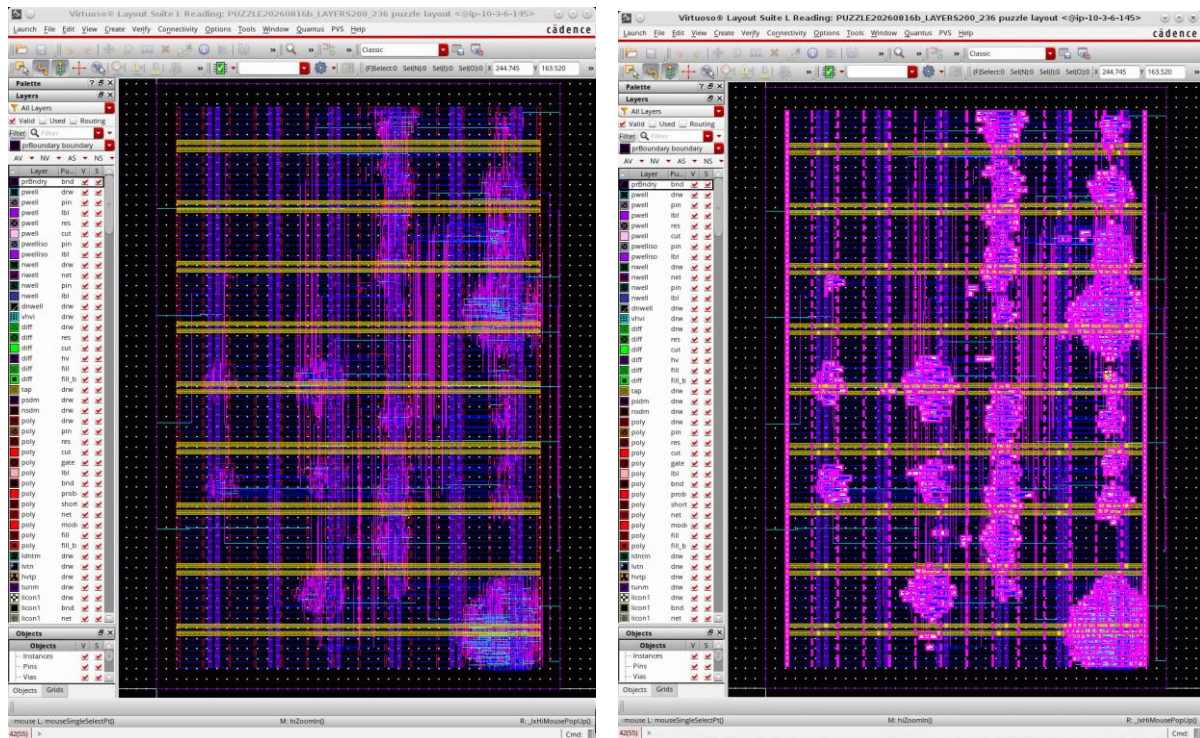
The next thing to do was to stream in the GDSII file provided within the puzzle repo. I started with the warmup (*04_final.gds*) before proceeding with the puzzle itself (*puzzle.gds*).

Streaming in was straightforward. In the Cadence Virtuoso software there is a menu item which can be used to import a GDSII stream so it was just a matter of creating a library for the incoming GDSII and importing the GDSII into said library. Note that the libraries have to have the *sky130_fd_pr_main* technology library attached to them, to be able to see the correct IC layers. Further note that the “layermap table” file *sky130_fd_pr_main.layermap* – which tells the software which GDSII layer number corresponds to which PDK layer – is automatically populated by the PDK scripts – see the screenshots below:

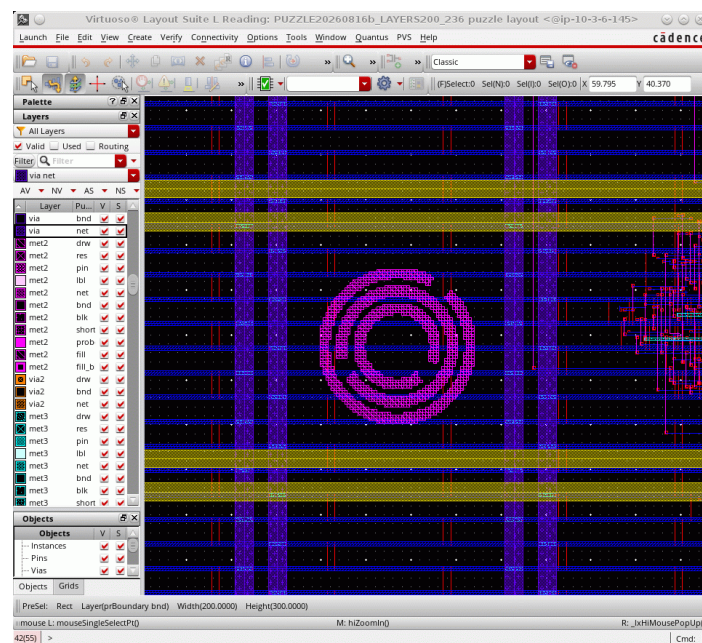


The stream-in was successful albeit with two warnings (see below), and the resulting layout “made sense” in that the wells, diffusion, and poly combined created what were obviously logic gates – i.e., they were in the correct layers, and there had been no mistranslation of layers during the stream-in process. This therefore vindicated my educated guess that the design would be in the Skywater 130nm process and its associated PDK. I further noted that the drawn gate length is 150nm which aligns with a 130nm process node – it would not be unusual for the drawn gate length to be different from the process node name, to more easily

align with a uniform design grid. The layout of the puzzle after streaming in is shown below in two views – one with the standard cell layouts empty, another with them filled out with their internal layouts:



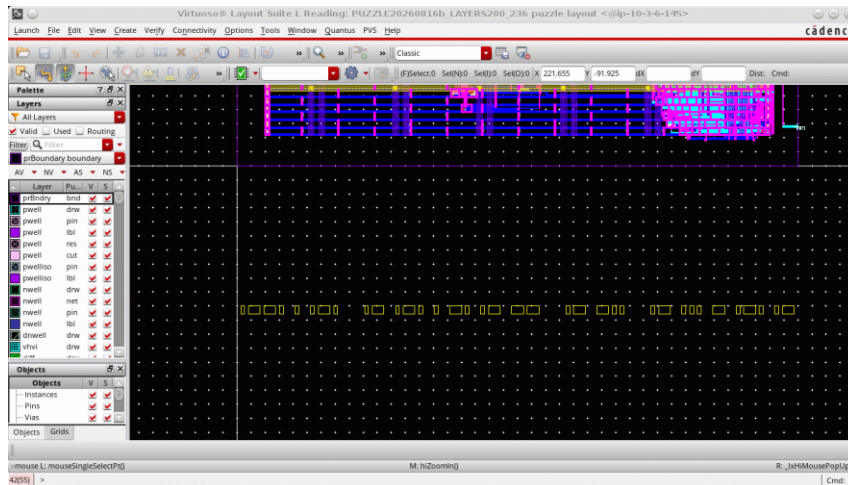
I noticed that the logo had been drawn in MET2 on the lower-left of the puzzle (which was also the case with the warmup albeit in that case in the upper-right). I doubt that this is an “Easter Egg” as it is so obvious but have shown it below nonetheless. Something fun to consider for the future might be to put a hologram of the logo using a diffraction pattern on the upper metal layers of the IC – there are scripts available to do this.



Mystery Layers in the GDSII

One thing that I noticed when streaming in the GDSII using the default layermap table was that a warning was issued about two layers being present in the GDSII which weren't in the layermap table – namely, layers 200:0 and 236:0, the former layer (200:0) being associated with a row of instantiated cells named “INTERNAL_3” and “INTERNAL_7” at the bottom of the design, which were currently empty. I therefore modified the layermap table file to stream these two layers into unused layers in the PDK (y0 and y1) such that I could view what was on them, as I suspected that they were either an Easter Egg or a hint, or both – especially in light of the hint given in the blog: “The circuit is physically arranged to hint at its functionality, so look closely at the layout!”

Layer 236:0 was a red herring – it just outlined the standard cells – but layer 230:0 was highly relevant, as it populated the formerly empty cells “INTERNAL_3” and “INTERNAL_7” at the bottom of the design with design data, namely what appeared to be some morse code – see the figure below:



The morse code is ".-.. .-. / .- .-. .-. -- / .- .. / .- ... - .-. .-" which reads as “PER ARENAM AD ASTRA”. Google suggested that this was a play on the (Latin) phrase “per aspera ad astra” which means “through hardships to the stars”. By substituting *aspera* for *arenam* the phrase “per arenam ad astra” appears to translate as “through the arena to the stars”. It would seem that, not only is this an Easter Egg, but it is hinting towards what the puzzle’s function might actually be – I will return to this later.

Finding the Netlist from the Layout

Extraction of the Layout to yield a SPICE Netlist

The next step was using the layout to yield a SPICE/CDL netlist. This can be done with an *extraction* tool along with an associated set of rules inside the PDK itself. The extraction tool for which I had a license available was Cadence’s *Physical Verification System* or PVS – this is

the forerunner of the more modern “Pegasus” Physical Verification Platform which was developed for today’s more advanced process nodes. Unfortunately, the Cadence PDK came with a rules deck for Pegasus, and **not** for PVS – however, the format of the two are similar and almost interchangeable, so it was just a case of slightly “hacking” the Pegasus rules to make them compatible with PVS. Furthermore, the only PVS tool I had to run was “Layout vs Schematic” or LVS, so that was the only rules file that I had to modify in the end. Using the modified rules file, I thus ran LVS in “extraction only” mode (rather than using it to compare the extracted netlist to a schematic, which of course I didn’t have) to yield a SPICE/CDL netlist, an excerpt of which is shown below (for the puzzle although note that this was performed first on the warmup):

```
* ++++++ *
* CDL Netlist: *
* * *
* Cell Name : puzzle *
* Netlisted : Sun Aug 16 17:47:18 2026 *
* PVS Version: 24.12-s015 Fri May 2 18:07:08 PDT 2025 *
* ++++++ *
...
* ++++++ *
* Sub cell: sky130_fd_sc_hd__and3_2 *
* ++++++ *
.subckt sky130_fd_sc_hd__and3_2 A B C VGND VPWR X VNB VPB
** N=30 EP=8 FDC=10
M0 12 A 9 VNB nfet_01v8 L=1.5e-07 W=4.2e-07 $X=410 $Y=265 $dt=8
M1 13 B 12 VNB nfet_01v8 L=1.5e-07 W=4.2e-07 $X=770 $Y=265 $dt=8
M2 VGND C 13 VNB nfet_01v8 L=1.5e-07 W=4.2e-07 $X=1175 $Y=265 $dt=8
M3 X 9 VGND VNB nfet_01v8 L=1.5e-07 W=6.5e-07 $X=1780 $Y=235 $dt=8
M4 VGND 9 X VNB nfet_01v8 L=1.5e-07 W=6.5e-07 $X=2200 $Y=235 $dt=8
M5 VPWR A 9 VPB pfet_01v8_hvt L=1.5e-07 W=4.2e-07 $X=405 $Y=1555 $dt=9
M6 9 B VPWR VPB pfet_01v8_hvt L=1.5e-07 W=4.2e-07 $X=825 $Y=1555 $dt=9
M7 VPWR C 9 VPB pfet_01v8_hvt L=1.5e-07 W=4.2e-07 $X=1300 $Y=1485 $dt=9
M8 X 9 VPWR VPB pfet_01v8_hvt L=1.5e-07 W=1e-06 $X=1795 $Y=1485 $dt=9
M9 VPWR 9 X VPB pfet_01v8_hvt L=1.5e-07 W=1e-06 $X=2215 $Y=1485 $dt=9
.ends sky130_fd_sc_hd__and3_2
...
* ++++++ *
* Sub cell: puzzle *
* ++++++ *
.subckt puzzle I 0[0] 0[1] 0[2] 0[3] 0[4] 0[5] 0[6] 0[7] VGND
+ VPWR clk enable rst_n success
** N=808 EP=15 FDC=10596
X8897 VGND VPWR VGND 714 sky130_fd_sc_hd__decap_3 $T=10120 282880 0 0 $X=9930
$Y=282640
X8898 VGND VPWR VGND 714 sky130_fd_sc_hd__decap_3 $T=10120 288320 1 0 $X=9930
$Y=285360
X8899 VGND VPWR VGND 715 sky130_fd_sc_hd__decap_3 $T=189980 282880 1 180
$X=188410 $Y=282640
X8900 VGND VPWR VGND 715 sky130_fd_sc_hd__decap_3 $T=189980 288320 0 180
$X=188410 $Y=285360
X8901 18 VPWR VGND 716 VGND VPWR sky130_fd_sc_hd__clkbuf_4 $T=57500 146880 00
$X=57310 $Y=146640
X8902 29 VPWR VGND 717 VGND VPWR sky130_fd_sc_hd__clkbuf_4 $T=70380 108800 1
0 $X=70190 $Y=105840
```

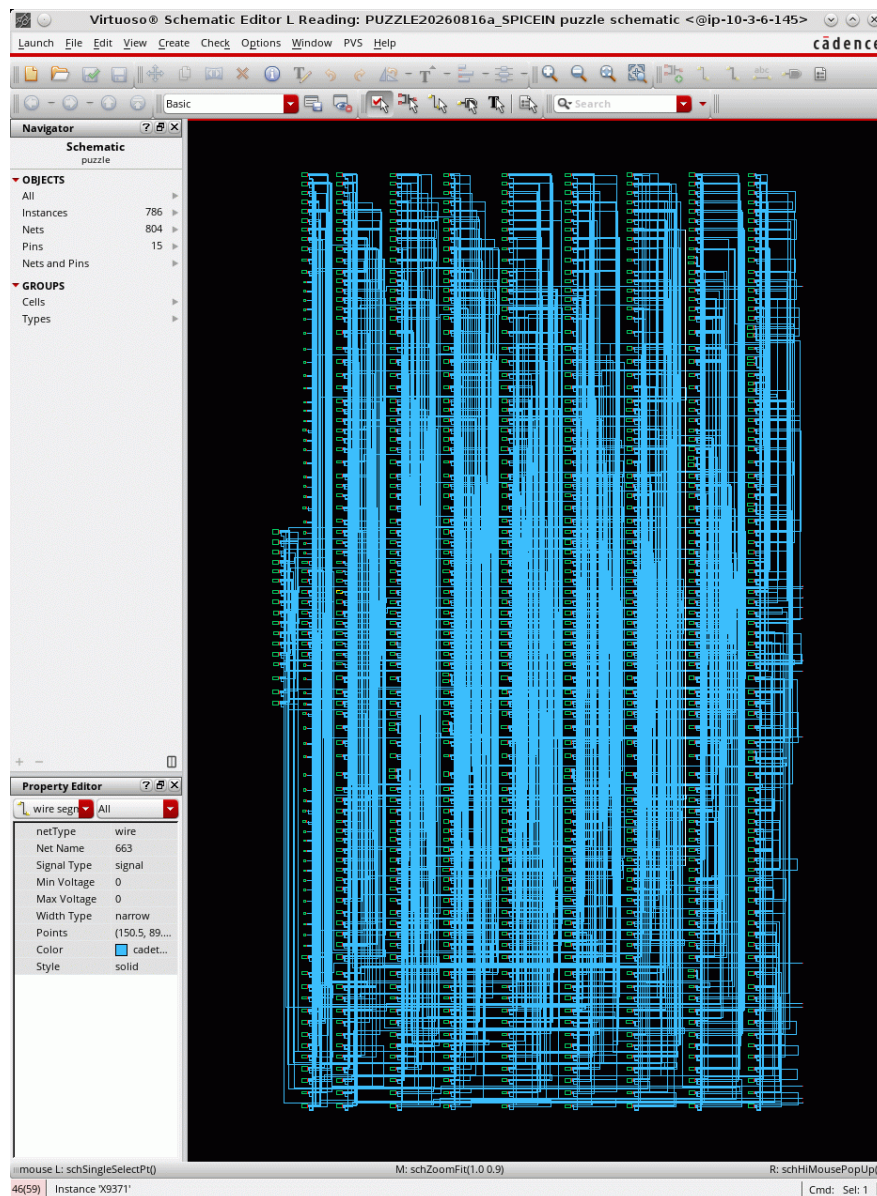
```
X8903 83 VPWR VGND 718 VGND VPWR sky130_fd_sc_hd__clkbuf_4 $T=77740 141440 1
180 $X=74790 $Y=14120
```

One good thing about the extracted SPICE/CDL netlist is that, while it went all the way down to individual transistors, it wasn't "flattened" and preserved the circuit hierarchy – i.e., the digital standard cells were preserved as individual .subckts – this made the next step, converting the SPICE netlist into a Verilog netlist, considerably easier.

However, one thing to note is that the standard cells in the resulting netlist are taken from the *open-source* version of the Skywater PDK and **not** the Cadence version that I installed – this will be covered in the next section.

Out of interest, I imported the SPICE netlist back into the Cadence Virtuoso Schematic Edit such that I could visualise it, with the resulting schematic shown below. Of course, automatically-generated schematics of netlists are rarely "human-readable", as can be seen.

Note that the Navigator on the left-hand side reports 786 instances, 804 nets, and 15 pins (including the power pins).



Conversion of the SPICE Netlist to a Verilog Netlist

The straightforward way of creating a Verilog Netlist from a SPICE netlist within an all-Cadence toolset environment is to use Cadence's Spice->Verilog translation tool. However, this requires a separately licensed product (the Virtuoso Simulation Interface) for which I didn't have a license available. There are other means within Cadence to do this translate which result in a Verilog**AMS** netlist but this would have meant I could not use Xcelium for simulation. Given how straightforward the netlists were, with the digital standard cells contained within .subckts as mentioned above, I decided to approach this particular step by making use of a perl script (spice2verilog_9t_composite.pl), aided by Chat GPT for expediency.

This was relatively straightforward, as the standard cell names had been preserved in the SPICE netlist and so I could just stop at the SPICE .subckt level and replace each .subckt with a Verilog module corresponding to a standard cell. I had to change the net names a bit to avoid escape characters and to make it a little more "human-readable". An excerpt of the resulting

netlist (of the puzzle; the warm-up was done prior to this to pipeclean the flow) is shown below:

```
/* Generated from puzzle.spi */
/* mode=skywater */

module puzzle (
    I,
    O_0_,
    O_1_,
    O_2_,
    O_3_,
    O_4_,
    O_5_,
    O_6_,
    O_7_,
    VGND,
    VPWR,
    clk,
    enable,
    rst_n,
    success
);

    input I;
    output O_0_;
    output O_1_;
    output O_2_;
    output O_3_;
    output O_4_;
    output O_5_;
    output O_6_;
    output O_7_ ;
    input VGND;
    input VPWR;
    input clk;
    input enable;
    input rst_n;
    output success;

    wire n_10, n_100, n_101, n_102, n_103, n_104, n_105, n_106, n_107, n_108,
n_109, n_11, n_110, n_111, n_112, n_113, n_114, n_115, n_116, n_117, n_118,
n_119, n_12, n_120, n_121, ...

    sky130_fd_sc_hd__a31o_2 X8942 (
        .A3(n_57),
        .A2(n_11),
        .A1(I),
        .B1(n_87),
        .X(n_136)
    );

    sky130_fd_sc_hd__a31o_2 X8943 (
        .A3(n_44),
        .A2(n_64),
        .A1(n_52),
        .B1(n_46),
        .X(n_73)
    );
};
```



```

sky130_fd_sc_hd__a31o_2 X8944 (
    .A3(n_44),
    .A2(n_64),
    .A1(n_62),
    .B1(n_66),
    .X(n_55)
);
...
sky130_fd_sc_hd__o31ai_2 X9638 (
    .A1(n_347),
    .A2(n_495),
    .A3(n_653),
    .B1(n_433),
    .Y(n_634)
);

endmodule

```

As alluded to above, this netlist uses the open-source PDK's standard cells, not those inside the Cadence PDK which are both named differently and consist of a different set of functions, presumably to be better-aligned with the expectations of Cadence synthesis and/or physical design tools such as Genus and/or Innovus, respectively.

I did attempt to convert the above netlist employing the open-source PDK standard cells into one that was functionally the same but using the Cadence PDK standard cells but it proved to be a challenging problem that would likely be helped enormously by insight into the overall top-level functionality of the circuit which of course, it being a puzzle, I did not have. Hence I decided to abandon this approach and proceeded with the above netlist using the open-source PDK's standard cells. This of course required that I download and install the open-source PDK's standard cells [4] which I proceeded to do – see the next section regarding the issues I had with using these with Xcelium for digital simulation.

Digital Simulation

I used Cadence's Xcellium simulator (xrun on the command line) with the zero delay (-nospecify) option as I was just testing the functionality of the circuit rather than examining its detailed timing. I first began by checking that the behaviour of the warmup puzzle's source Verilog (00_source.v) matched that of the gate-level netlist (01_netlist) and came across some problems covered in the next section (note, some of the problems below were discovered later when I came to simulate the puzzle as well but are kept together in one section for clarity).

Standard Cell Simulation Issues

The first problem I had to tackle was how to get the open-source skywater standard cells working with the Cadence Xcelium simulator. This required considerable "hacking" to get these to work with said simulator; presumably they work OK with the open-source simulation

tools but some of the problems were so serious I am mystified as to how. The main difficulties in using the models with Cadence Xcelium were:

1. The standard cell library isn't a "flat" Verilog library and in particular the "top-level" standard cells are simply wrappers of cells deeper in the hierarchy. This isn't unusual, and was resolved with the help of Chat GPT to construct a makefile (make_sky130_f.csh) that created a single *sky130_fd_sc_hd.f* library file containing all cells and include paths, in the correct order, that could be included on the Xcelium command-line with 'xrun -f sky130_fd_sc_hd.f'
2. Invalid Verilog in the standard cells. For example, *sky130_fd_sc_hd__dlxbn.behavioural.v* contains the line:

```
wire 1 ;
```

which is invalid Verilog. This was removed.
3. The behavioural flip-flop models contained undriven nets, resulting in the flip-flops outputting "X" values which then propagate throughout the simulation. E.g., *sky130_fd_sc_hd__dftrp.behavioral.v* contains:

```
wire D_delayed ;  
wire RESET_B_delayed;  
wire CLK_delayed ;
```

And only *RESET_B* is actually driven within that module.

The 2nd and 3rd problems led me to ultimately use the functional, rather than the behavioural, models – since I was only interested in ascertaining the function of puzzle.v rather than the precise details and veracity of its timing, this was acceptable. Note that I set the `UNIT_DELAY define within these models to be #0 – meaning zero delay – from the xrun command line as I only wanted to inspect function, not timing.

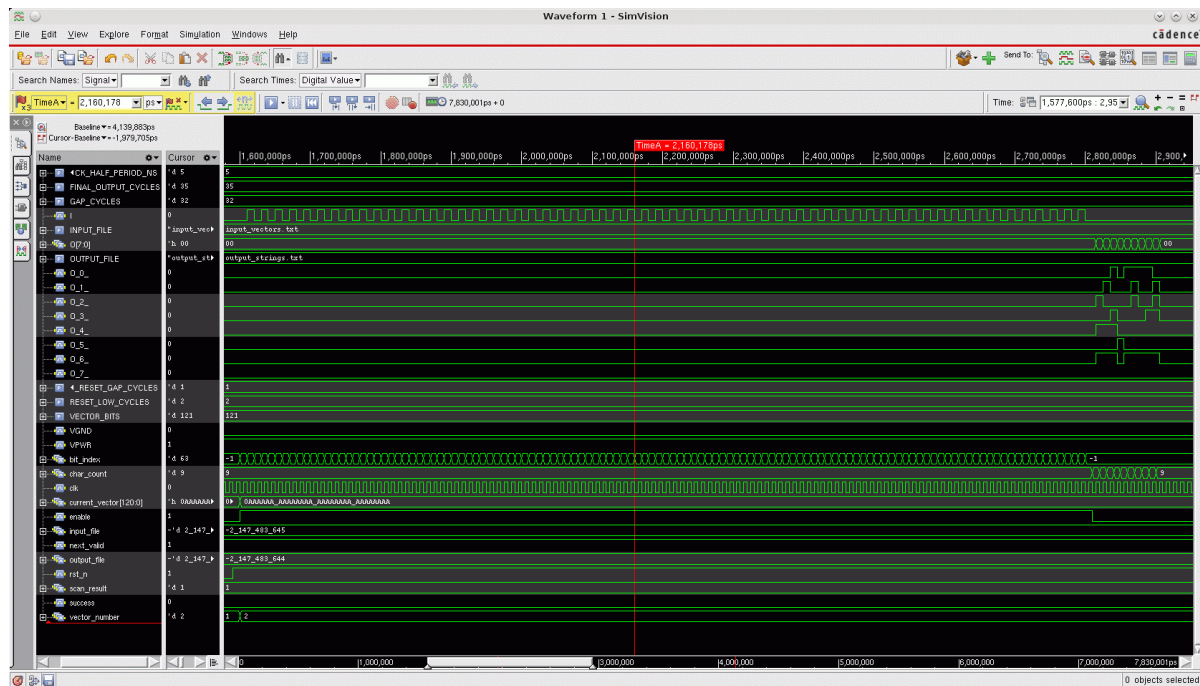
There were a few other easily resolvable problems:

1. The "bleeder" cells which are just weak open-circuit "stubs" caused compilation problems – since these were not included in the designs, I just removed them from the *sky130_fd_sc_hd.f* library file
2. The design contained some *MASCO** cells which had no verilog models associated with them. These looked to simply be filler or decoupling cells with no actual logic function associated with them and I simply deleted them manually from the netlist

Finally, with all the above done, I could compile, elaborate, and run a simulation with the following *xrun* command-line options (to run in GUI) mode. Note that *-nospecify* is used, as I had no interest in examining detailed timing, merely function. Further note that *puzzle_tb7inoutfile.v* is the testbench, which I will discuss later.

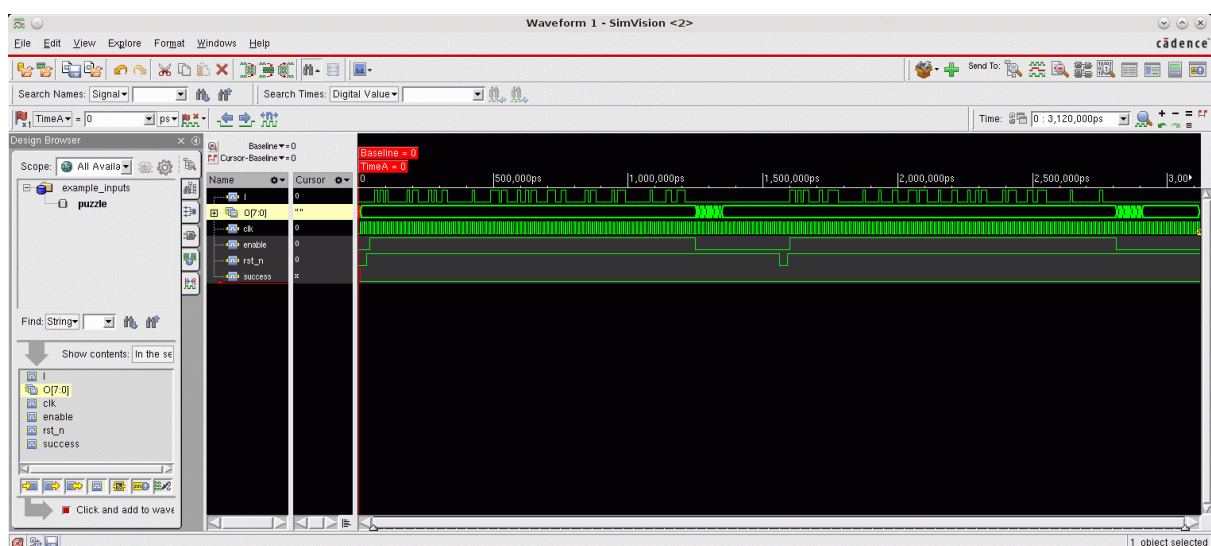
```
xrun -sv -timescale 1ns/1ps -access +rwc -nospecify -linedebug -gui -define  
FUNCTIONAL -define 'UNIT_DELAY=#0' -f sky130_fd_sc_hd.f puzzle.v  
puzzle_tb7inoutfile.v
```

An example output of SimVision arising from the above command showing salient waveforms is given below.



Example Inputs VCD File

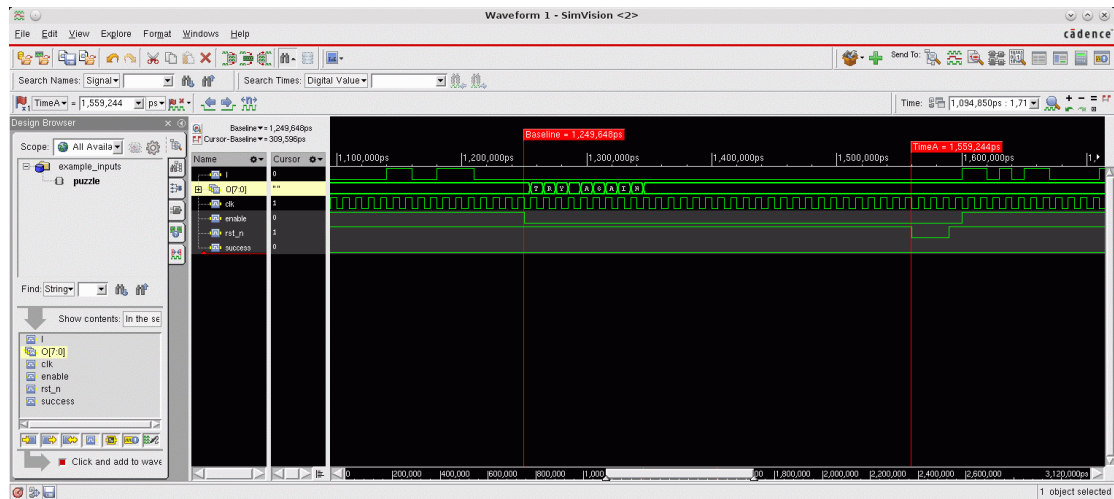
The next question to address was how to drive the *puzzle.v* circuit. Thankfully, an example .vcd file *example_inputs.vcd* was provided, showing how to stimulate the circuit, and it was straightforward to read this file into Cadence SimVision such that the stimuli could be examined: see the figure below:



Several things can be observed:

- The *rst_n* input is held low for 3 clock cycles to reset the flops
- One clock cycle later, the *enable* input is asserted

- The *enable* input is held high for 121 clock cycles, presumably whilst the *I* input is clocked in
 - This gives a strong hint as to the nature of the puzzle
- The serial *I* input changes on clock falling edges, thus suggesting it is clocked in on clock rising edges, which is usual practice
- The *enable* input is deasserted and held low for 31 clock cycles during which the parallel output *O*[7:0] is clocked out revealing the ASCII text “TRY AGAIN” – see below



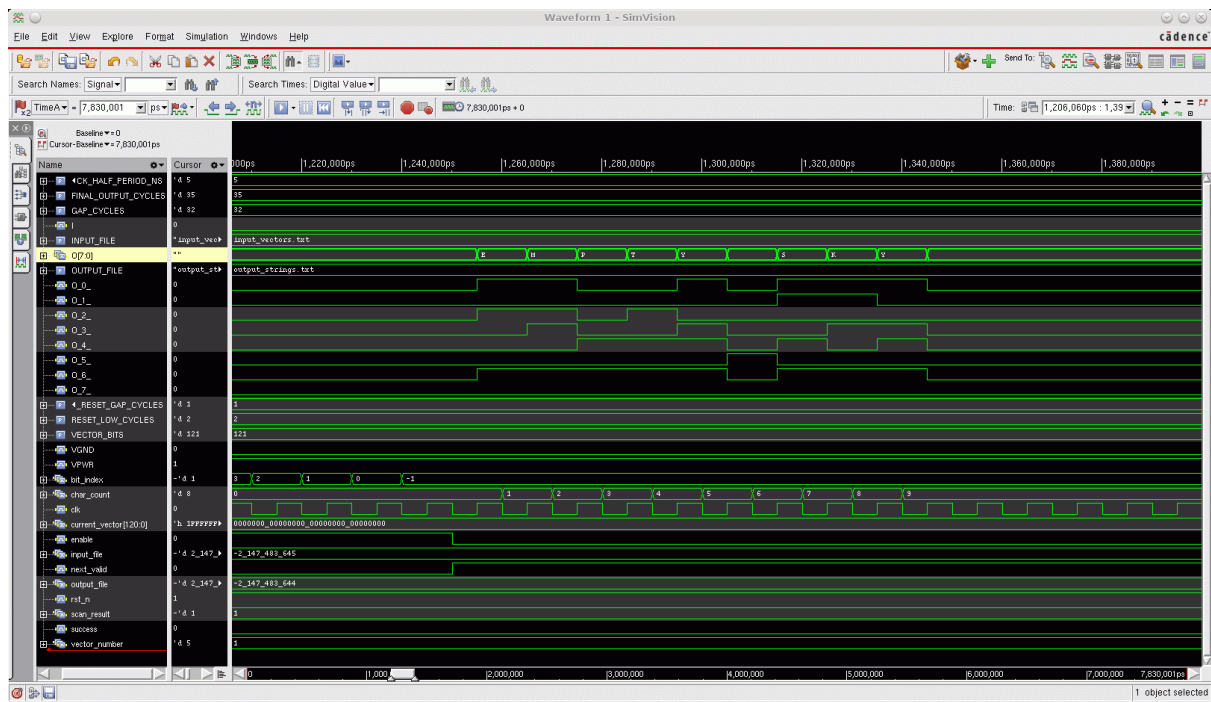
- Further note that the output changes on rising clock edges and thus would want to be sampled downstream on *falling* clock edges
- After this, the *rst_n* input is once again driven low and the cycle repeats

Verilog Testbench Creation

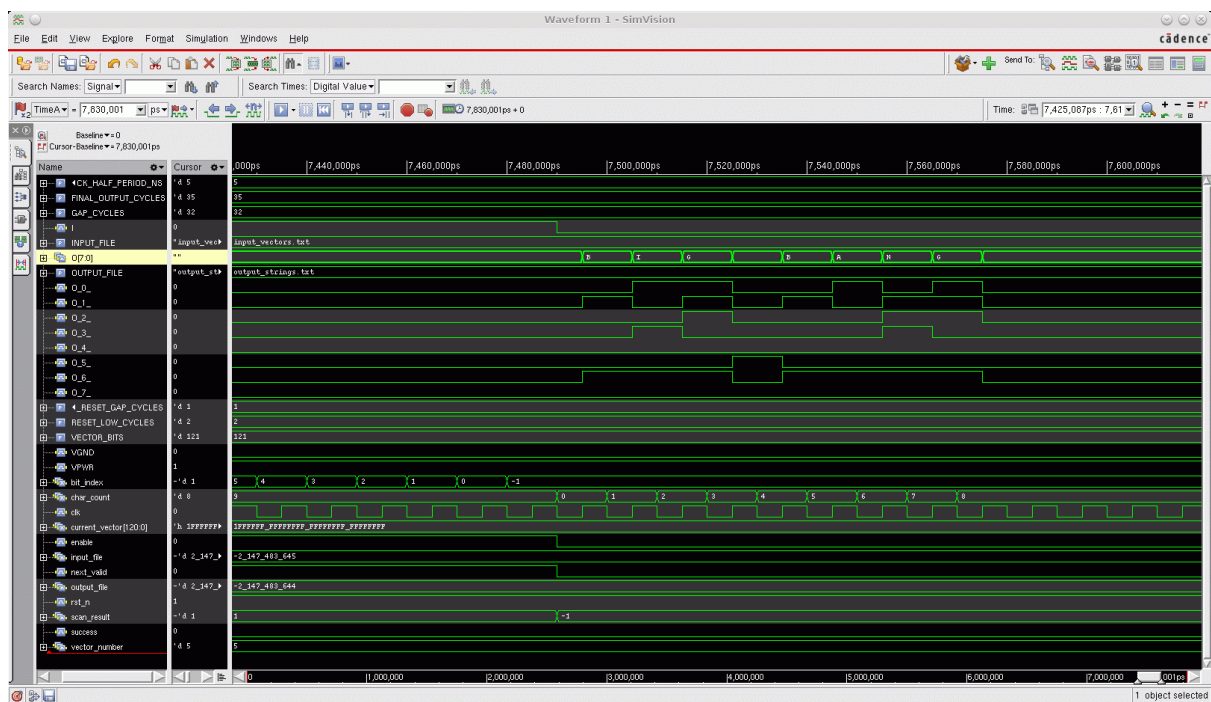
The above analysis of the *example_inputs.vcd* file gave me enough information to be able to create a Verilog testbench for *puzzle.v*, which I decided to do. I made all the timings parameterised such that, e.g., the length of the input vector and the length of the output string, could be easily changed. Furthermore, it was clear that I would want to try multiple input vectors as trial solutions, and also would want to save the resulting output strings. I thus devised a testbench that would read input vectors from a file *input_vectors.txt* and write the resulting output strings to an output file *output_vectors.txt*. See the testbench file *puzzle_tb7inoutfile.v* in the repo. The simulation with the testbench can be run with the Xcelium command:

```
xrun -sv -timescale 1ns/1ps -access +rwc -nospecify -linedebug -gui -define
FUNCTIONAL -define 'UNIT_DELAY=#0' -f sky130_fd_sc_hd.f puzzle.v
puzzle_tb7inoutfile.v
```

Whilst experimenting with this testbench, I accidentally noticed that when I clocked in a vector that was all zeros, '0000...' that the output text was "EMPTY SKY" – see below:



This piqued my curiosity, and I was curious what an input vector of all ones, ‘1111...’ would yield and found that the output text was “BIG BANG” – see below:



I tried a few other guesses including “010101...”, “1010101...” and the words “Jane Street” as ASCII in various combinations of capitals, lower-case, and bit/byte order to no avail.

The fact that “0000...” gives the text “EMPTY SKY” and “1111...” gives “BIG BANG” are both, I believe, Easter Eggs. Furthermore, I believed that they were clues as to what the puzzle did – this is covered in the next section.

Finding the Solution

Figuring Out the Puzzle's Function

I now had multiple hints as to what the function of the puzzle might be:

- 121 input bits being clocked in
 - Notably, this is 11^2 , possibly suggestive of a square grid of input bits
- The “hidden” morse code text in the GDSII on layer 230:0, ".-- . . - / . - . . . - . - - / . - - . / . - ... - . - . -" which reads as “PER ARENAM AD ASTRA” and translates as “through the arena to the stars”.
- The words “EMPTY SKY” being output when a vector of all 0’s is clocked in
- The words “BIG BANG” being output when a vector of all 1’s is clocked in

It was fairly straightforward to work out from the above hints that the puzzle was, most likely, a **Star Battle** puzzle on an 11x11 grid.

I started at this point to endeavour to trace the logic back from the ‘success’ output by probing nets in SimVision’s “Schematic Tracer” tool to see if that would yield a solution but with the circuit standing at 736 standard cell instances and 805 nets, it quickly became apparent that this approach would take a substantial amount of time to do. Hence I decided to adopt a more “brute force” approach based on guesswork as described in the next section – this was done “at risk”, as there was always the possibility that I was incorrect about the puzzle being a Star Battle puzzle. Note, however, that the initial work I did on tracing the logic back was not entirely wasted – for example, I found evidence that the success output was a combinational AND of 4 inputs, one of which appeared to come from a count of 121 input bits, and another of which counted if two bits in a row were high. These observations added some credence to the deductions above.

Trial Input Vectors (Perl)

Given that I had designed my Verilog testbench to read input vectors from a text file, I decided that the next step was to create a text file of input vectors containing *all possible* solutions to an 11x11 grid Star Battle puzzle, in which regions were not specified (as obviously I did not know anything about the regions). This was straightforward to do in Perl with the help of Chat GPT. I made the script general such that it could look for solutions to puzzles with any number of stars and could, optionally, have regions specified at a later date via an input file. See the script *starbattle_vectors_general.pl* in the repo. I began with looking for 2-star solutions, as this is the most common format for this puzzle, by running the script as follows:

```
perl starbattle_vectors_general.pl --stars 2 --input_vectors.txt
```

This generated 31,197,434 possible 2-star solutions, for a regionless 11x11 Star Battle grid.

Digital Simulation (Cadence Xcelium)

I quickly checked that it was feasible to simulate all ~31M possible solutions by running a few trials – it was, by using 16 cores on an AWS r6i.8xlarge instance. Hence I ran a simulation in which all ~31M possible solutions were tested, running Xcelium in non-GUI mode with the following options:

```
xrun -sv -timescale 1ns/1ps -nospecify -define FUNCTIONAL -define  
'UNIT_DELAY=#0' -f sky130_fd_sc_hd.f puzzle.v puzzle_tb7inoutfile.v
```

This took about 6 hours of simulation time, running overnight. I could then easily test if a solution had been found using grep:

```
grep -a -vn 'TRY AGAIN' output_strings.txt
```

Which pleasingly gave the output:

```
29300056:(* TWO STARS *)
```

Thus giving the **final answer**, the output string:

```
(* TWO STARS *)
```

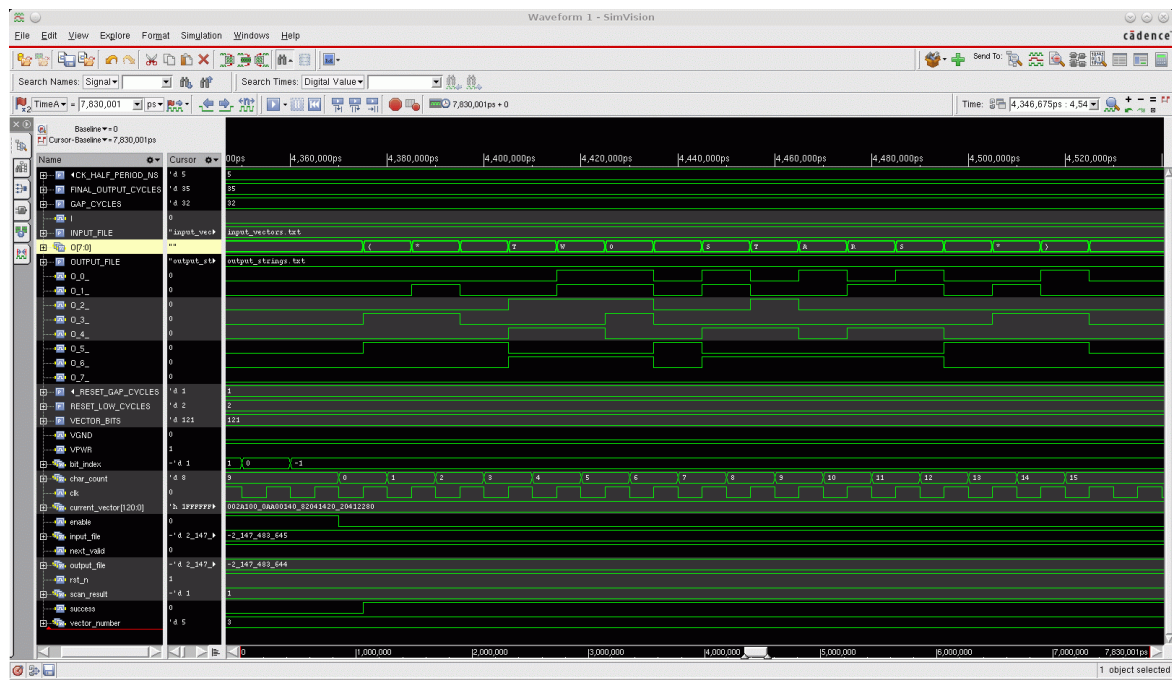
It was then relatively straightforward to find the corresponding successful input vector, just by looking at line 29300056 in input_vectors.txt:

```
sed -n '29300056p' input_vectors.txt
```

To give the input vector corresponding to the solution as:

```
000000010101000010000000000000101010100000000000010100000010000010000001000001  
010000100000000100000010000010010001010000000
```

Waveforms demonstrating that the solution was indeed obtained are given in the SimVision screenshot below – you can see the text “(* TWO STARS *)” on the O[7:0] output.

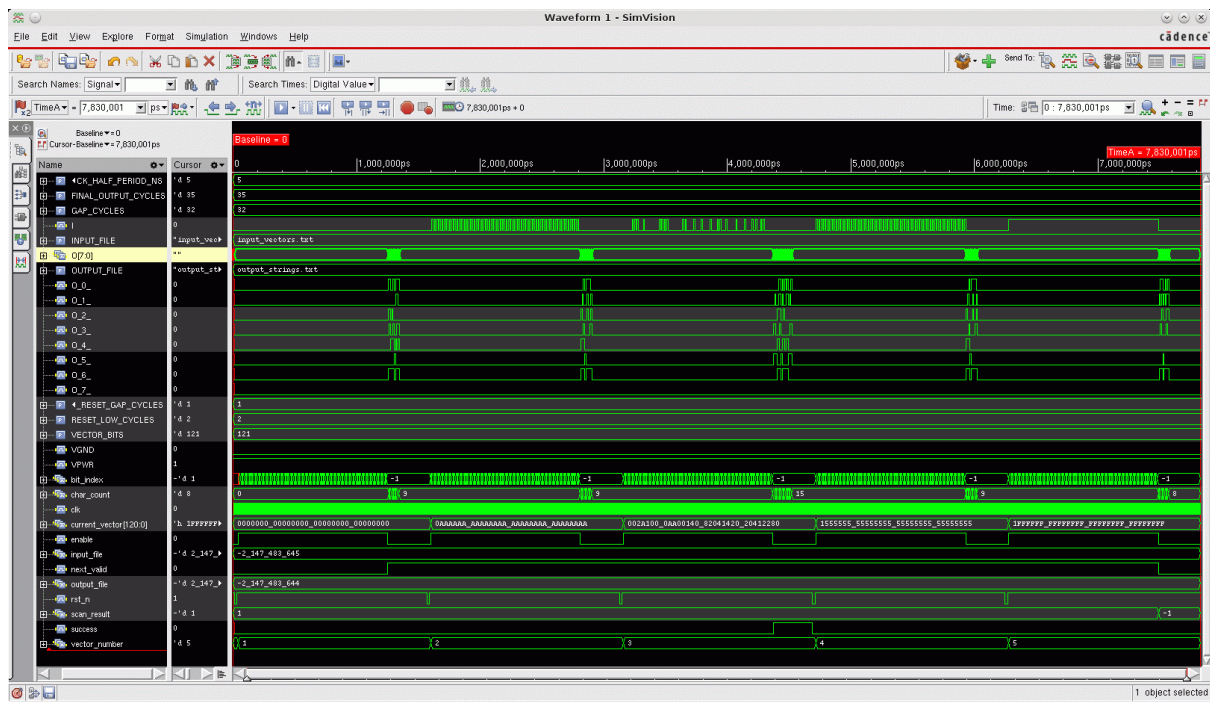


I then checked that there were no 1-star, or 3, 4, or 5-star solutions (anything above 5 does not make sense on an 11x11 grid). I confirmed that 3, 4, and 5-star solutions were mathematically impossible with the help of Chat GPT. Regarding 1-star solutions, I ran my perl script again but this time with the '-- stars 1' option to give 5,296,794 possible solutions on a regionless grid and re-ran the Verilog testbench again in non-GUI mode. No output strings other than "TRY AGAIN" were found, confirming that no 1-star solutions were present.

To complete this activity, and by way of double-checking, I then created a much simpler *input_vectors.txt* file that included the 2-star solution above but "topped and tailed" by vectors consisting of 0000..., 0101..., and 1010..., 1111... which gave an *ouput_strings.txt* file as follows:

```
EMPTY SKY
TRY AGAIN
(* TWO STARS *)
TRY AGAIN
BIG BANG
```

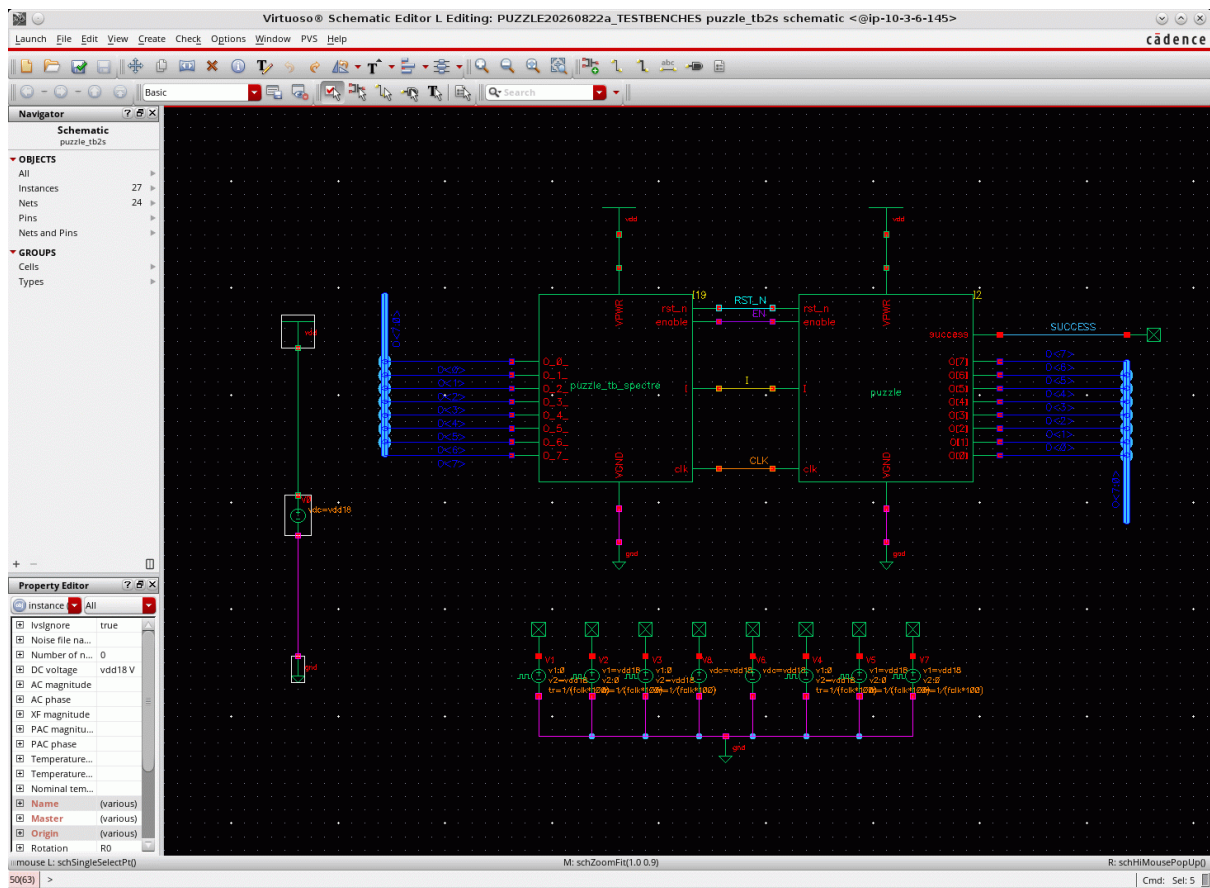
Which was exactly as-expected. The SimVision screenshot of the entire simulation is shown below.



Analog Simulation (Cadence Spectre)

Since I had access to the Cadence analog design tools (the Virtuoso schematic editor, the Spectre circuit simulator, and various other tools associated with both) and had already created a SPICE netlist for the puzzle, I thought it would be worthwhile to do a full-transistor analog simulation of the design, as a “belt and braces” check of functionality as well as out of general interest.

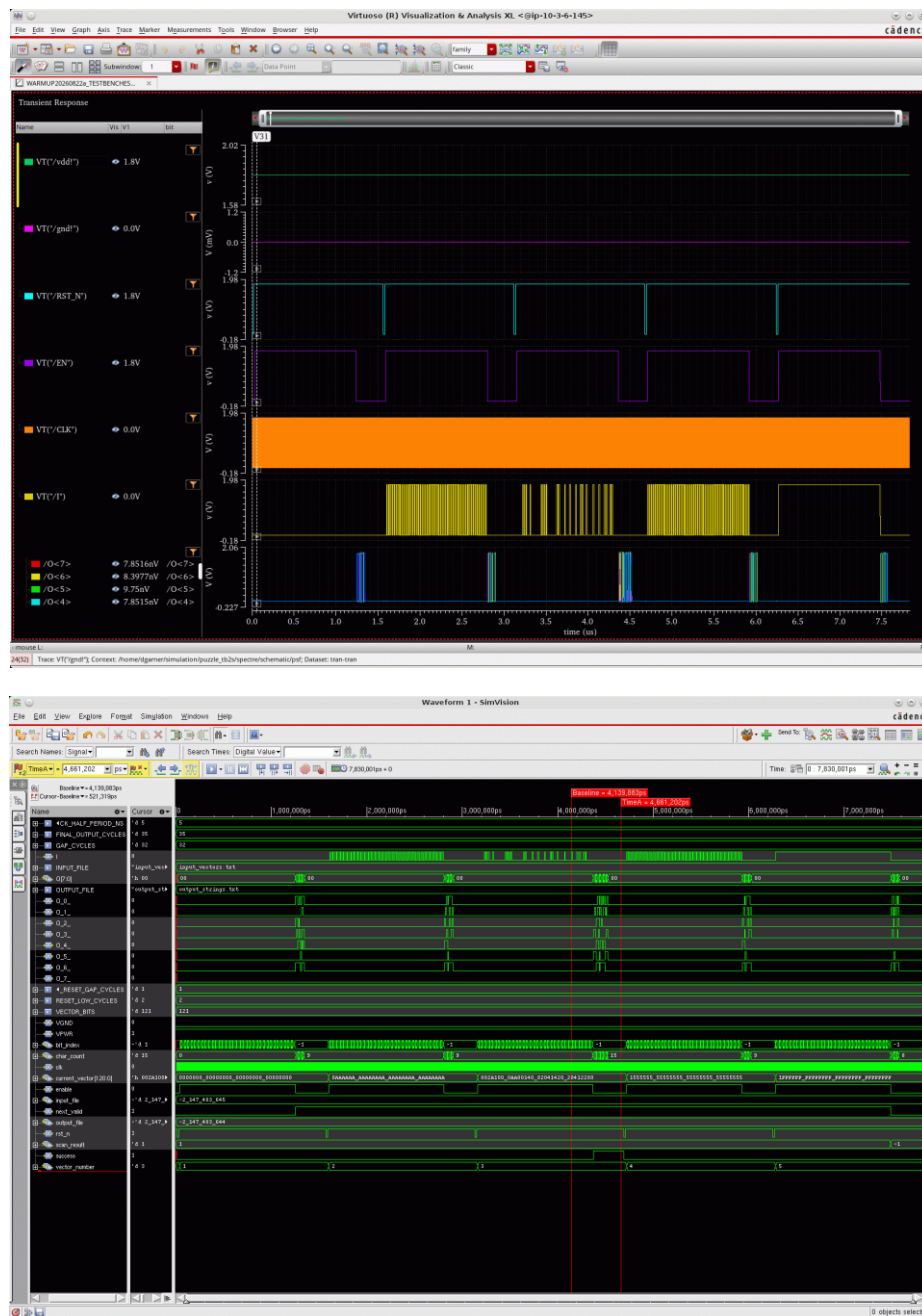
I converted my Verilog testbench into a VerilogA testbench such that it could be used with “pure” analog-only Spectre simulations (as opposed to needing analog-digital cosimulation with Cadence’s AMS simulator) and created a test schematic in which both were hooked up as shown below:



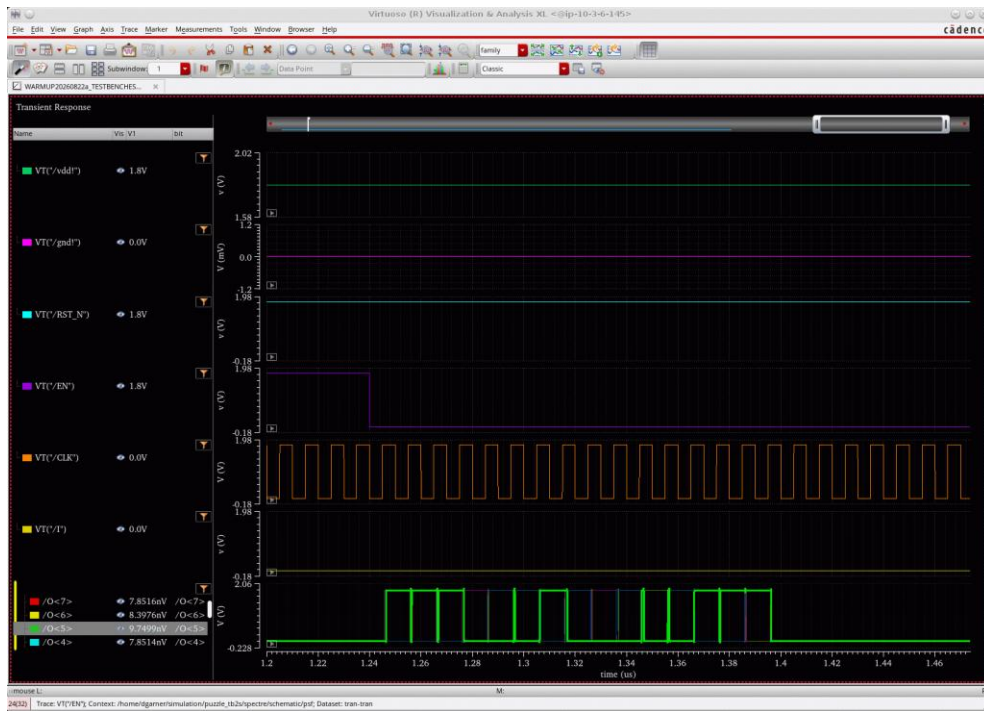
The testbench (shown above instantiated centre-left “puzzle_tb_spectre”) used the same *input_vectors.txt* file to create the input stimuli and, in a similar fashion to the digital simulations, output the results to a file *output_strings.txt*. Because the compute time required for analog simulations is much greater than that for digital simulations, I only simulated my simple “top and tail” input vectors (consisting of 0000..., 0101..., the solution vector, 1010..., 1111...) which yielded the same set of output strings as with the digital simulation, to wit:

EMPTY SKY
 TRY AGAIN
 (* TWO STARS *)
 TRY AGAIN
 BIG BANG

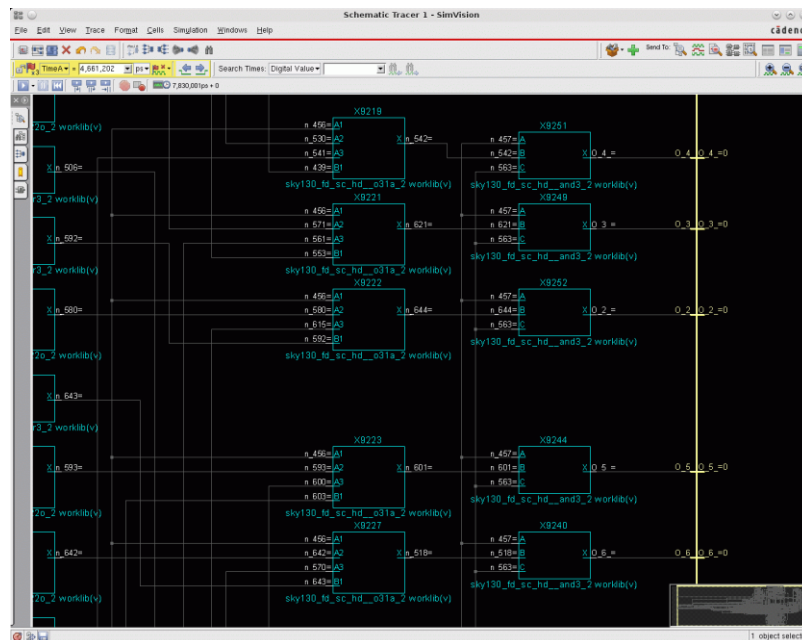
Thus the digital simulation results were verified. The output waveforms from the analog simulations are shown below, above those from the digital simulations with identical input stimuli and timing. Their equivalency can be seen.



The analog simulations did reveal one interesting difference compared to the digital simulations, however. With the digital simulations I was ignoring timing; with analog simulations, there is not this luxury as they are modelling the real analog behaviour of each individual transistor. What the analog simulations revealed were the presence of large glitches on the outputs between successive '0' bits and successive '1' bits, as shown in the example below:



These glitches may or may not be problematic in the end application, depending on how the O[7:0] output is used: as long as it is sampled on falling clock edges, it will be OK. The glitches arise because the outputs are driven directly from combinational logic gates, as shown below (in the digital Schematic Tracer tool rather than the analog Schematic Editor tool, as the former is clearer):



If the glitches on the outputs were problematic with a downstream application, they could be removed easily by registering the outputs by using, e.g., latched D-type flops, but this would be at the expense of additional latency of up to a full clock cycle, which might be intolerable. Normally this would be handled by setting constraints at synthesis accordingly.

Conclusion

In summary, I have reverse-engineered the layout of the Jane Street puzzle that was available only as layout information in GDSII format. I did this using the Cadence suite of IC Design tools – no open-source tools were used at all – along some custom perl scripts and a Verilog testbench. I mainly used the Cadence PDK for this project, only reverting to the open-source PDK for the digital standard cells.

The **answer** is:

(* TWO STARS *)

For an input vector of:

```
00000000101010000010000000000000101010100000000000000101000000100000100000010000001
0100001000000001000000100000100100010100000000
```

Regarding the “Easter Eggs”, it rather depends on what is classed as an Easter Egg! But things that I found that weren’t necessary as part of the main puzzle solution were:

1. The Jane Street logo on the MET2 layer on the layout (although this was hardly hidden, so it is unlikely to be an Easter Egg)
2. A hidden layer in the GDSII, layer 230:0, that revealed the morse code text ".--. . -. / .- .-. .-. -- / .- -. / - ... - -. .-" which reads as “PER ARENAM AD ASTRA” in Latin, translating to English as “through the arena to the stars”
3. An output string of “EMPTY SKY” for an input vector consisting solely of zeros
4. An output string of “BIG BANG” for an input vector consisting solely of ones

Contact Details

If you would like to discuss any aspects of my solution, please feel free to email me on david@davidgarner.org.uk or ‘phone or message on +44 7734 435186. Also please see my LinkedIn page below [6].

Appendix: List of Scripts / Testbenches

These will all be available on my GitHub after the puzzle submission date [5].

1. Customised layer map table file for “revealing” layer 230:0 on which the “Easter Egg” morse code was written: *sky130_fd_pr_main.layermap_200y0_236y1*
2. Perl script to convert a SPICE netlist into a Verilog netlist:
spice2verilog_9t_composite.pl

3. “Makefile” for compiling all the open-source standard cells into a single file for inclusion as a library in xcelium. This needs to sit at the top of the standard-cell installation hierarchy: *make_sky130_f.csh*
4. Verilog testbench for digital simulations of the puzzle Verilog: *puzzle_tb7inoutfile.v*
5. Perl script to generate valid Star Battle puzzle solutions for an 11x11 grid: *starbattle_vectors_general.pl*
6. Simple input and resulting output files with the solution input vector “topped and tailed” to give 0000...,0101...,solution,1010,1111:
input_vectors_solution_toptail20260821a.txt and
output_strings_solution_toptail20260821a.txt

References

- [1] Jane Street Blog Post detailing the “Can You Reverse Engineer an ASIC?” puzzle: <https://blog.janestreet.com/can-you-reverse-engineer-an-asic/>
- [2] Jane Street Repository containing the puzzle files: <https://github.com/janestreet/asic-puzzle-2026>
- [3] Cadence Skywater 130nm (sky130) Process Design Kit (PDK) – note, only available to those with a Cadence Support Account: <https://ask.cadence.com/ASK/article-viewer?id=a1Od000000051TqEAI&pageName=article-viewer&shareID=0611f3574ceb>
- [4] Open-Source Skywater 130nm (sky130) Process Design Kit (PDK): <https://github.com/google/skywater-pdk>
- [5] My GitHub containing the customised layer map table file, “makefile” for the standard cells, all developed perl scripts, the Verilog testbench and example input vectors with associated output strings (available only after the puzzle submission deadline of 4/9/2026): <https://github.com/davidg351/Jane-Street-Puzzle-August-2026>
- [6] My LinkedIn page: <https://www.linkedin.com/in/david-garner-41a84621/>